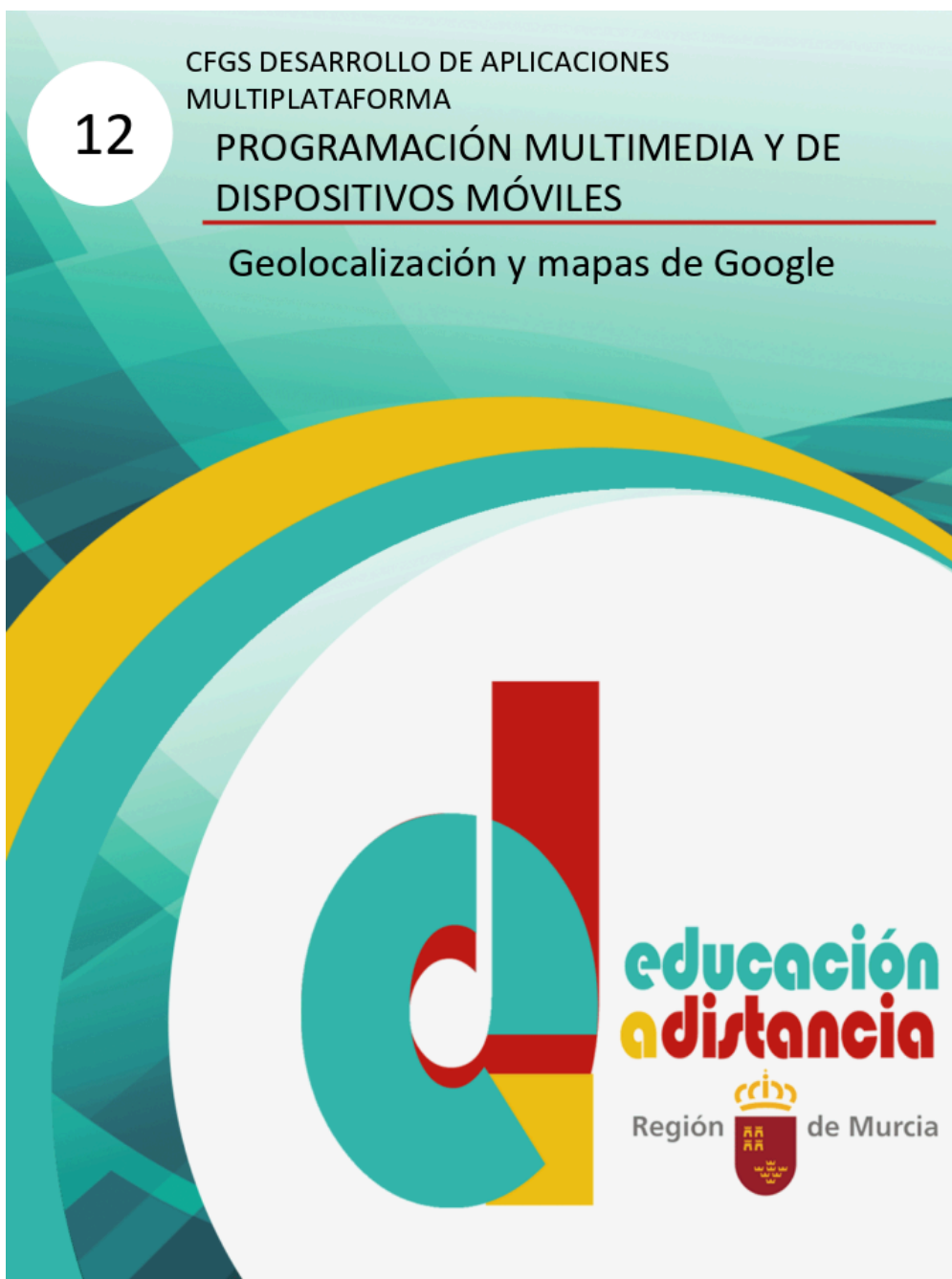


PMDM12.- Geolocalización y mapas de Google.

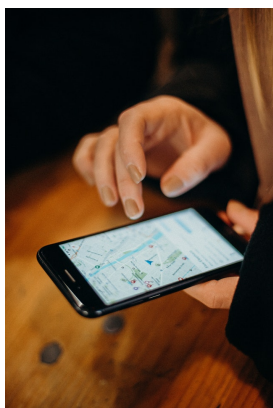


[Ministerio de Educación y Formación Profesional](https://eadfp.murciaeduca.es/pluginfile.php/117070/mod_resource/content/3/singlepage_index.html). (Dominio público)

Materiales formativos de FP Online propiedad del Ministerio de Educación y Formación Profesional.

[Aviso Legal](#) 

Caso práctico



[cottonbro](#) (Pexels)

El cliente de BK Programación que solicitó la aplicación para sus compradores de bicicletas, que tomaba valores a partir de los sensores de los dispositivos Android, desea incorporar algunas funcionalidades más relacionadas con el tracking de rutas.

- ¿Os acordáis del cliente que vendía bicicletas? -pregunta Ada.

- ¡Cómo no! -responde Pedro-, aprendimos un montón sobre sensores con ese pedido.

- Ahora quiere incorporar el tracking de rutas -informa Ada-, y creo que lo más adecuado es utilizar los mapas y la tecnología de Google Maps.

Incorporar esta tecnología supone usar la API **Maps SDK for Android** que Google ofrece a los desarrolladores y a las empresas de forma gratuita hasta un umbral o número de veces en que el mapa se muestra a los usuarios finales. Si mensualmente se supera este umbral, Google realizará cobros por entender que los usuarios o los intermediarios usan la aplicación Google Maps de forma comercial.

- Comenzaremos registrando una cuenta en **Google Cloud** -explica Ada-, y configurando un proyecto donde activemos la **Maps SDK for Android** para desarrolladores de Android. Sólo queda aprender cómo crear fragmentos, actividades y elementos de la pantalla que sean capaces de interactuar con la aplicación y apoyarse en el servicio de mapas de Google.

-¡Comencemos! -exclama el equipo al unísono.

1.- Geolocalización.

Caso práctico



[Caleb Oquendo](#) (Pexels)

- Estamos viendo -informa Juan a Ada- que para realizar un tracking de rutas es necesario hacer geolocalizaciones.
- Muy bien visto -felicit a Ada-, y ¿qué ocurre cuando una aplicación quiere acceder a nuestra ubicación?
- Que debemos darle permisos -responde Juan-. Vamos a investigar sobre las características de esos permisos.

Una de las funciones importantes de las aplicaciones móviles es la detección de ubicaciones. Los usuarios de plataformas móviles llevan consigo sus dispositivos a todas partes. A su vez, cuando se agrega la detección de ubicaciones se ofrece a los usuarios una experiencia más contextual.

Sin embargo, como desarrolladores de aplicaciones debemos ser conscientes de la importancia de la privacidad de los datos de nuestros usuarios. En este elemento concreto, la ubicación de los usuarios puede ser un elemento muy importante a proteger, por ello las aplicaciones que usan servicios de ubicación deben solicitar permisos para ello. Por este motivo debemos solicitar a nuestros usuarios que nos conceda el permiso adecuado que necesitemos.

Cada permiso tiene una combinación de dos características, la **categoría** o modo en que se ejecuta el acceso a la ubicación (en **primer plano** o en **segundo plano**) y la **precisión** de la ubicación (precisión **aproximada** o **precisa**).

1.1.- Tipos de acceso a la ubicación.

Categoría ubicación en primer plano

Se usa cuando la ubicación es necesaria en algún momento concreto. Por ejemplo, cuando una aplicación de mensajería necesita compartir la ubicación del usuario. En este caso es necesario declarar en el archivo de manifiesto una entrada de servicio indicando "location".

Código

```
1 <service
2     android:name="MyNavigationService"
3     android:foregroundServiceType="location" ... >
4     <!-- Any inner elements would go here. -->
5 </service>
```

El sistema considera que la aplicación usa esta categoría si:

- ✓ Una Activity de la aplicación que usa "ubicación" es visible.
- ✓ La aplicación ejecuta un servicio en primer plano que se mantiene activo incluso cuando la aplicación no es la que está como principal, o incluso la pantalla está apagada.

En cuanto a la precisión, es posible especificar si necesitamos ubicación precisa o ubicación predeterminada. En ambos casos, en el archivo de manifiesto deberemos indicar cuál de los permisos es necesario.

Categoría ubicación en segundo plano

Una aplicación requiere acceso a la ubicación en segundo plano si una función dentro de la aplicación comparte constantemente la ubicación con otros usuarios o usa la [API de geovallado](#) . Por ejemplo, una aplicación familiar donde se comparte la ubicación de alguno de los miembros con otros.

El sistema considera que tu aplicación usa ubicación en segundo plano si hace un uso diferente al indicado en primer plano. Para poder usar este tipo de aplicación es necesario indicar en el archivo de manifiesto el siguiente permiso:

Código

```
1 <manifest ... >
2   <!-- Required only when requesting background location access on
3       Android 10 (API level 29) and higher. -->
4   <uses-permission android:name="android.permission.ACCESS_BACKGROUND_LOCATION" /
5 </manifest>
```

Debes conocer

Google Play Store tiene una [política sobre la ubicación del dispositivo](#)  en la que el acceso a la ubicación en segundo plano se restringe a las aplicaciones que lo necesiten para su funcionalidad principal, y cumplan los requisitos de políticas relacionadas.

1.2.- Tipos de precisión.

Android admite los siguientes niveles de precisión de la ubicación:

Aproximada

Proporciona una estimación de la ubicación del dispositivo. Si esta estimación de la ubicación es de LocationManagerService o FusedLocationProvider, la estimación es exacta en un radio de 3 kilómetros cuadrados. Tu aplicación puede recibir ubicaciones con este nivel de precisión cuando declaras el permiso ACCESS_COARSE_LOCATION, pero no el permiso ACCESS_FINE_LOCATION. Android usa datos de Wi-Fi o datos móviles para determinar la ubicación del dispositivo.

Precisa

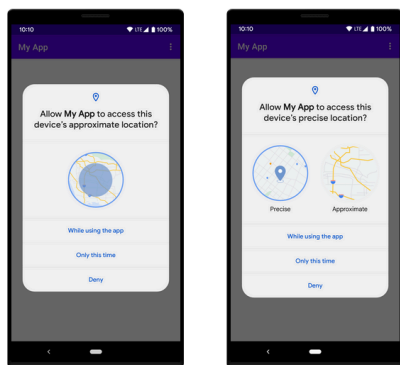
Proporciona una estimación de la ubicación del dispositivo que sea lo más precisa posible. Si la estimación de la ubicación es de LocationManagerService o FusedLocationProvider, suele ser de 50 metros y, a veces, tan precisa como de unos pocos metros. Tu aplicación puede recibir ubicaciones en este nivel de precisión cuando declaras el permiso ACCESS_FINE_LOCATION. Android usa el sistema de posicionamiento global (GPS) y los datos de Wi-Fi y los datos móviles para determinar la ubicación del dispositivo.

El permiso que elijamos determina la precisión de la ubicación devuelta por la API. Según el nivel de precisión que necesitemos, solo debes solicitar uno de los permisos de ubicación de Android en el archivo de manifiesto.

Ejemplo

```
1 <manifest ... >
2   <!-- Always include this permission -->
3   <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
4
5   <!-- Include only if your app benefits from precise location access. -->
6   <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
7 </manifest>
```

1.3.- Solicitud de permisos en tiempo de ejecución.



[Google Developers](#) (Uso educativo no)

Cuando una función de tu aplicación necesite acceso a la ubicación, espera hasta que el usuario interactúe con ella para solicitar el permiso.

En Android 12 (nivel de API 31) o versiones posteriores, los usuarios pueden solicitar que tu aplicación solo recupere información de ubicación aproximada, incluso cuando tu aplicación solicite el permiso de ubicación precisa en tiempo de ejecución. Se recomienda en general solicitar sólo la ubicación aproximada.

Para ofrecer esta elección al usuario, la aplicación debe solicitar en el archivo de manifiesto `ACCESS_FINE_LOCATION` y `ACCESS_COARSE_LOCATION`. En este caso, el diálogo de permisos del sistema incluirá ambas opciones para el usuario, como puede verse en la segunda captura de la imagen de esta página.

Observa que, además, el usuario puede elegir si desea otorgar el permiso mientras la aplicación está en uso (siempre que esté en uso), sólo para esta ocasión, o denegarlo.

Autoevaluación

El sistema considera que la aplicación usa la ubicación en primer plano cuando...

- ☐ La aplicación ejecuta un servicio en primer plano que se mantiene activo incluso cuando la aplicación no es la que está como principal o incluso la pantalla está apagada.

- ☐ Una función dentro de la aplicación comparte constantemente la ubicación con otros usuarios.

- ☐ Una Activity de la aplicación que usa "ubicación" es visible.

Mostrar retroalimentación

Solución

1. Correcto
2. Incorrecto
3. Correcto

2.- Mapas de Google.

Caso práctico

- Ya tenemos claro el funcionamiento de los permisos para que una aplicación pueda realizar geolocalizaciones -informa María a Ada.
- ¡Genial! -exclama Ada-. Ahora vamos a realizar una pequeña práctica con la tecnología de Google Maps para afianzar esos conocimientos.



[Brett Jordan \(Pexels\)](#)

Con la API **Maps SDK for Android**, puedes agregar mapas a tu aplicación para Android, incluidas las aplicaciones para Wear OS que utilizan datos, reproducciones de mapas y respuestas gestuales de Google Maps. También puedes ofrecer información adicional sobre las ubicaciones del mapa y facilitar la interacción con el usuario si agregas marcadores, polígonos y superposiciones a tu mapa.

El SDK es compatible con los lenguajes de programación Kotlin y Java, y ofrece bibliotecas y extensiones adicionales para funciones avanzadas como agregar:

- ✓ Iconos anclados en posiciones específicas del mapa (veremos en el ejercicio resuelto el uso del método **addMarker()** para crear marcadores en el mapa).
- ✓ Conjuntos de segmentos de líneas (polilíneas).
- ✓ Segmentos cerrados (polígonos).
- ✓ Gráficos de mapa de bits anclados en posiciones específicas del mapa (marcadores).
- ✓ Conjuntos de imágenes que se muestran sobre los mosaicos de mapas básicos (superposiciones de mosaicos).

2.1.- Ejercicio resuelto 1 (mapas de Google Maps y geolocalización).

Debes conocer

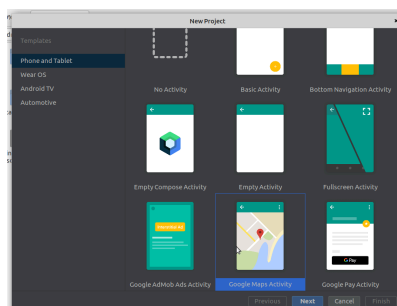
Para llevar a cabo este ejercicio es necesario que consigas una "API Key" que te permita interactuar con los servicios de Google. Para ello es necesario contar con una cuenta de **correo de Google y una tarjeta de crédito válida**.

Todos los ejercicios se pueden llevar a cabo sin coste, ya que Google, en el momento de escribir este contenido, ofrece un periodo de prueba gratuito de 90 días o 300\$. Este crédito es más que suficiente para llevar a cabo los ejercicios propuestos. Esta cuenta de prueba puede cancelarse una vez transcurrido el tiempo de la realización de estas prácticas. Digamos que, para poder activar la API Key Google, comprueba que el proyecto está asociado a una cuenta de facturación correcta, aunque son necesarias miles de visitas para que se cobre por ello. Esto está orientado para empresas que se apoyan en el servicio de Google Maps para ofrecer sus servicios.

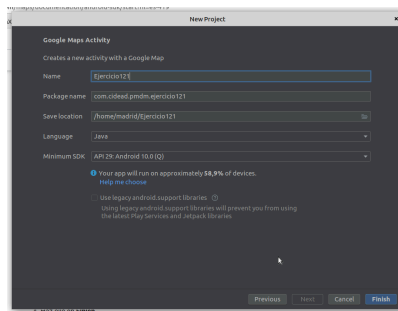
En caso de no disponer de tarjeta de crédito, debes contactar con el tutor del curso para salvar este problema, probablemente él pueda compartir una APY Key con propósitos educativos.

Se trata de crear una aplicación en la que agregues un mapa mediante el **Maps SDK for Android** y lo muestres con una ubicación. Como EXTRA trabajaremos para que muestre la ubicación actual de tu terminal.

1.- El primer paso es crear un proyecto de Google Maps en Android Studio ([más ayuda](#) ).



[Google Developers](#). (Uso educativo nc)



[Google Developers](#). (Uso educativo nc)

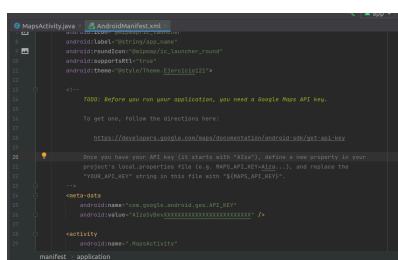
Esta acción terminará habiendo creado el proyecto y mostrará dos ficheros abiertos sobre los que trabajar, el archivo de manifiesto y **MapsActivity.java**. Si observas, se ha creado un archivo **activity_maps.xml** donde se despliega un fragmento para albergar el mapa de Google a pantalla completa.

2. El segundo paso consiste en conseguir una API Key de Google Maps. Para ello tendrás que registrar tu cuenta de correo en [Cloud console de Google](#) , crear un proyecto nuevo, crear una cuenta de facturación donde asignes datos de facturación (incluso una tarjeta de crédito obligatoria) y finalmente activar la API **Maps SDK for Android**. Es posible que el nombre del servicio y la forma de hacerlo varíe en el tiempo. Deberás encontrar información de cómo hacerlo en Internet. En el momento de escribir este contenido, puedes encontrar ayuda de cómo hacerlo en este [enlace](#) .

3. El siguiente paso es añadir nuestra API Key al proyecto. Para ello lo más sencillo es seguir las indicaciones de Android Studio y completar la variable android:value del elemento <meta-data> con tu API key (figura 1).

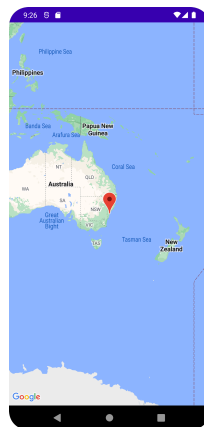
El resultado es una aplicación que abre el fragmento de Google Maps con el mapa ubicado en Sidney (Australia), como se muestra en la figura 2.

Figura 1



[Google Developers](#). (Uso educativo nc)

Figura 2



[Google Developers](https://developers.google.com/edu/nc/) (Uso educativo nc)

Si observas el código generado automáticamente en el archivo **MapsActivity.java**, podrás observar que se ha programado el evento **onMapReady()** para que, cuando esté preparado, se actualice y se añada un marcador (la tachuela roja de siempre) mediante la función **addMarker()**. Finalmente, se centra la posición del mapa en el marcador mediante el método **moveCamera()** pasando el objeto que guarda las coordenadas. También es posible cambiar el aspecto del marcador.

MapsActivity.java

```

1 | @Override
2 | public void onMapReady(GoogleMap googleMap) {
3 |     mMap = googleMap;
4 |
5 |     // Add a marker in Sydney and move the camera
6 |     LatLng sydney = new LatLng(-34, 151);
7 |     mMap.addMarker(new MarkerOptions().position(sydney).title("Marker in Sydn
8 |     mMap.moveCamera(CameraUpdateFactory.newLatLng(sydney));
9 | }

```

Elije unas coordenadas de latitud y longitud de un lugar que te guste (tu pueblo, tu ciudad, un sitio que te guste) y actualiza la línea de código dónde se crea el objeto de la clase `LatLng`. Prueba de nuevo la aplicación y verifica que el mapa se abre las coordenadas indicadas.

```

1 | LatLng sydney = new LatLng(Latitud elegida, Longitud elegida);

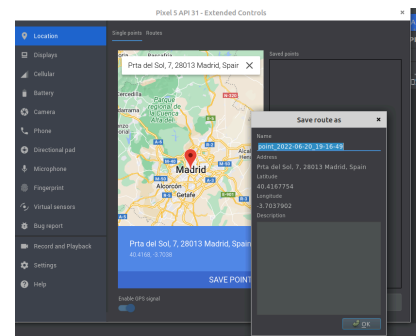
```

4. APARTADO EXTRA: El último paso consiste en cambiar esta ubicación por defecto para que en realidad el mapa muestre nuestra ubicación. Puedes establecer la ubicación para los terminales emulados mediante la opción "Location" de los controles extendidos.

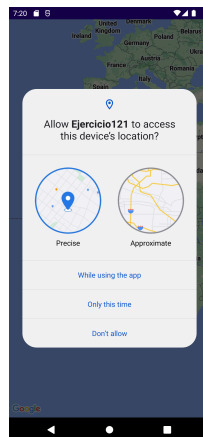
Te recomendamos que, después de establecer la localización mediante los controles extendidos, y antes de ejecutar la aplicación, abras Google Maps en el emulador y verifiques

que te reconoce la ubicación que hayas establecido como tu ubicación actual. De esta forma te aseguras que el terminal ya ha solicitado la ubicación y tiene algún valor al respecto.

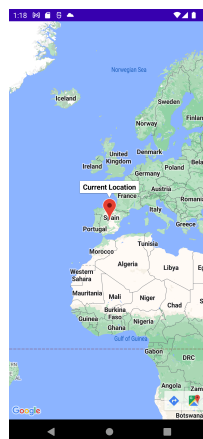
Ahora, al ejecutar la aplicación te solicitará los permisos y, finalmente, mostrará la ubicación elegida (en este caso, Madrid).



[Google Developers](#) (Uso educativo nc)



[Google Developers](#) (Uso educativo nc)



[Google Developers](#) (Uso educativo nc)

A continuación te mostramos el contenido de algunos archivos importantes.

AndroidManifest.xml

Permisos en **AndroidManifest.xml**.

```
1 <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
2 <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
3 <uses-permission android:name="android.permission.INTERNET"/>
4 <uses-permission android:name="android.permission.ACCESS_BACKGROUND_LOCATION" />
```

build.gradle(:app)

Dependencias en **build.gradle(:app)**.

```
1 dependencies {
2
3     implementation 'androidx.appcompat:appcompat:1.4.2'
4     implementation 'com.google.android.material:material:1.6.1'
5     implementation 'com.google.android.gms:play-services-maps:18.0.2'
6     implementation 'com.google.android.gms:play-services-location:20.0.0'
7     implementation 'androidx.constraintlayout:constraintlayout:2.1.4'
8     implementation 'org.jetbrains:annotations:16.0.2'
9
10    testImplementation 'junit:junit:4.13.2'
11    androidTestImplementation 'androidx.test.ext:junit:1.1.3'
12    androidTestImplementation 'androidx.test.espresso:espresso-core:3.4.0'
13
14 }
```

MapsActivity.java

Archivo **MapsActivity.java**.

```
package com.cidead.pmdm.ubicacion;

import androidx.annotation.NonNull;
import androidx.core.app.ActivityCompat;
import androidx.fragment.app.FragmentActivity;

import android.Manifest;
import android.content.pm.PackageManager;
import android.location.Location;
import android.os.Bundle;
import android.widget.Toast;

import com.google.android.gms.location.FusedLocationProviderClient;
import com.google.android.gms.location.LocationServices;
import com.google.android.gms.maps.CameraUpdateFactory;
import com.google.android.gms.maps.GoogleMap;
```

```

import com.google.android.gms.maps.OnMapReadyCallback;
import com.google.android.gms.maps.SupportMapFragment;
import com.google.android.gms.maps.model.LatLng;
import com.google.android.gms.maps.model.MarkerOptions;
import com.cidead.pmdm.ubicacion.databinding.ActivityMapsBinding;
import com.google.android.gms.tasks.OnSuccessListener;
import com.google.android.gms.tasks.Task;

public class MapsActivity extends FragmentActivity implements OnMapReadyCallback

    private GoogleMap mMap;
    private ActivityMapsBinding binding;
    Location currentLocation;
    FusedLocationProviderClient fusedLocationProviderClient;
    private static final int REQUEST_CODE=101;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        binding = ActivityMapsBinding.inflate(getLayoutInflater());
        setContentView(binding.getRoot());
        fusedLocationProviderClient = LocationServices.getFusedLocationProviderCl
        getCurrentLocation();
    }

    /**
     * Manipulates the map once available.
     * This callback is triggered when the map is ready to be used.
     * This is where we can add markers or lines, add listeners or move the camer
     * we just add a marker near Sydney, Australia.
     * If Google Play services is not installed on the device, the user will be p
     * it inside the SupportMapFragment. This method will only be triggered once
     * installed Google Play services and returned to the app.
     */
    @Override
    public void onMapReady(GoogleMap googleMap) {
        mMap = googleMap;

        LatLng tuUbicacion = new LatLng(currentLocation.getLatitude(),currentLoca
        mMap.addMarker(new MarkerOptions().position(tuUbicacion).title("Current L
        mMap.moveCamera(CameraUpdateFactory.newLatLng(tuUbicacion));
    }

    private void getCurrentLocation(){
        if (ActivityCompat.checkSelfPermission(this, Manifest.permission.ACCESS_F
            && ActivityCompat.checkSelfPermission(this, Manifest.permission.A
            ActivityCompat.requestPermissions(this, new String[]{Manifest.permiss
            return;
        }
        Task<Location> task = fusedLocationProviderClient.getLastLocation();
        task.addOnSuccessListener(new OnSuccessListener<Location>() {
            @Override
            public void onSuccess(@NonNull @org.jetbrains.annotations.NotNull Loc
                if (location != null) {
                    currentLocation = location;
                    Toast.makeText(MapsActivity.this, String.valueOf(currentLocat

```

```
76         SupportMapFragment supportMapFragment = (SupportMapFragment)
77         assert supportMapFragment != null;
78         supportMapFragment.getMapAsync(MapsActivity.this);
79     }
80 }
81 });
82 }
83
84 @Override
85 public void onRequestPermissionsResult(int requestCode, @NonNull String[] per
86     super.onRequestPermissionsResult(requestCode, permissions, grantResults);
87     switch (REQUEST_CODE) {
88         case REQUEST_CODE:
89             if (grantResults.length > 0 && grantResults[0] == PackageManager.
90                 getLocation();
91             }
92             break;
93     }
94 }
```

Es una tarea compleja. Puedes apoyarte en este vídeo para llevarlo a cabo:

<https://www.youtube.com/embed/cnlSyYeRqrs>

[Descripción textual del vídeo "Localización actual"](#) 

Puedes consultar más ayuda en esta guía rápida de creación de aplicaciones de [Google Maps para Android Studio](#). 

Para saber más

Google recomienda por seguridad otras opciones para añadir la API Key a la aplicación mediante local.properties. Aquí tienes [más ayuda para intentar esta otra opción](#)  para intentarlo.